
Projet :

Système de commande à distance de volets roulants

Elaboré par :
Taher Bouderbela
Rayen Abid
Sadek Dhokar
Arbi Mnasria

Département génie informatique industrielle

Année universitaire :2025/2026

Sommaire

1. [Introduction générale](#)
 2. [Présentation du domaine IoT \(Smart Home\)](#)
 3. [Objectifs et cahier des charges](#)
 4. [Architecture globale du système \(MQTT\)](#)
 5. [Implémentation logicielle \(Software\)](#)
 6. [Conception matérielle \(Hardware\)](#)
 7. [Expérimentation et tests](#)
 8. [Conclusion et perspectives](#)
 9. [Références](#)
 10. [Annexes](#)
-

Liste des figures

- Figure 1 : Interface de l'application – Page d'accueil (statut + liste des pièces)
- Figure 2 : Interface de l'application – Page de commande d'une pièce (HAUT/BAS + slider)
- Figure 3 : Extrait du code Flutter – Connexion MQTT et abonnement au statut
- Figure 4 : Extrait du code Flutter – Publication des commandes MQTT
- Figure 5 : Schéma/diagramme de l'architecture MQTT (Publisher/Broker/Subscriber)
- Figure 6 : Carte ESP32 utilisée
- Figure 7 : Montage de test (LEDs / charges, résistances, câblage)
- Figure 8 : Moniteur série – Connexion Wi-Fi réussie
- Figure 9 : Moniteur série – Connexion MQTT + subscriptions
- Figure 10 : Moniteur série – Réception de commandes (UP/DOWN/%)
- Figure 11 : Test MQTT (HiveMQ Web Client ou autre outil) – publication sur esp32/room1
- Figure 12 : Test boutons manuels (pins 18/19) – preuve de fonctionnement
- Figure 13 : Test du slider 0–100% – durée proportionnelle (0–10 s)

1. Introduction générale

Avec la démocratisation des réseaux sans fil, des microcontrôleurs à faible coût et des smartphones, l'**Internet des Objets (IoT)** s'est imposé comme une solution majeure pour automatiser, superviser et contrôler des systèmes physiques à distance. Dans le cadre des **maisons intelligentes (Smart Home)**, l'IoT permet par exemple de commander l'éclairage, les volets roulants, le chauffage, la sécurité, ou encore de suivre des paramètres environnementaux (température, humidité, consommation énergétique).

Dans ce projet, nous proposons un prototype permettant de commander des **volets roulants** (ou des équivalents de test) depuis un smartphone. L'objectif est de réaliser une chaîne complète :

- une **application mobile** (Flutter) servant d'interface utilisateur ;
- une carte **ESP32** chargée d'exécuter physiquement les commandes ;
- une communication réseau fiable et légère via **MQTT**.

Le résultat attendu est une solution fonctionnelle, testable, et présentable, mettant en évidence la complémentarité entre développement mobile, programmation embarquée et conception matérielle.

2. Présentation du domaine IoT (Smart Home)

2.1 Définition

L'IoT (Internet of Things) désigne un ensemble d'objets physiques capables de :

- collecter des données (capteurs),
- recevoir des commandes (actionneurs),
- échanger des informations via un réseau (Wi-Fi, Bluetooth, Ethernet, 4G/5G, etc.).

Ces objets communiquent avec des services (serveur local, cloud) afin de proposer des fonctionnalités automatisées et de supervision.

2.2 Intérêt du Smart Home

Dans une maison intelligente, l'IoT apporte :

- **confort** (contrôle à distance, scénarios),
- **sécurité** (capteurs intrusion, caméra, alertes),
- **économie d'énergie** (optimisation d'éclairage/chauffage),
- **accessibilité** (pilotage simplifié via smartphone).

2.3 Pourquoi MQTT en IoT

MQTT (*Message Queuing Telemetry Transport*) est un protocole de messagerie léger adapté à l'IoT. Il se distingue par :

- un modèle **Publish/Subscribe** (découplage entre émetteur et récepteur),

- une faible surcharge réseau,
 - une bonne compatibilité avec les microcontrôleurs,
 - la possibilité de gérer des niveaux de qualité de service (QoS) et des messages retenus (*retained messages*).
-

3. Objectifs et cahier des charges

3.1 Objectif général

Concevoir un système permettant de **commander des volets roulants** depuis une application mobile, en s'appuyant sur une carte ESP32 et une communication MQTT.

3.2 Objectifs spécifiques

- Développer une application Flutter proposant une interface simple : sélection de pièce et commandes.
- Établir une connexion MQTT vers un broker public (`broker.hivemq.com`).
- Publier des commandes de type :
 - "up" (monter),
 - "down" (descendre),
 - une valeur numérique 0 . . 100 (pourcentage).
- Programmer l'ESP32 pour :
 - se connecter au Wi-Fi,
 - se connecter au broker MQTT,
 - s'abonner aux topics des pièces,
 - agir sur des sorties GPIO (commande bidirectionnelle A/B).
- Prévoir un **contrôle manuel local** (boutons) pour assurer une forme de continuité de service.

3.3 Hypothèse de calibration (commande par pourcentage)

Pour relier un pourcentage d'ouverture à un mouvement réel, nous utilisons l'hypothèse suivante :

- **100% de course (ouverture complète) correspond à 10 s de fonctionnement.**

$$\text{move time} = \frac{\text{target percentage}}{100} \times 10000 \text{ ms}$$

Cette approximation est suffisante pour un prototype ; dans une solution industrielle, on ajouterait des capteurs de position ou des fins de course.

4. Architecture globale du système (MQTT)

4.1 Rôles

- **Application Flutter** : *publisher* des commandes + *subscriber* du statut.
- **ESP32** : *subscriber* des commandes + *publisher* de son statut.
- **Broker MQTT** : serveur intermédiaire assurant la distribution des messages.

4.2 Topics utilisés

- Statut ESP32 : `esp32/status`
- Commandes pièces :
 - `esp32/room1` (Chambre 1)
 - `esp32/room2` (Chambre 2)
 - `esp32/room3` (Salon)

4.3 Diagramme MQTT

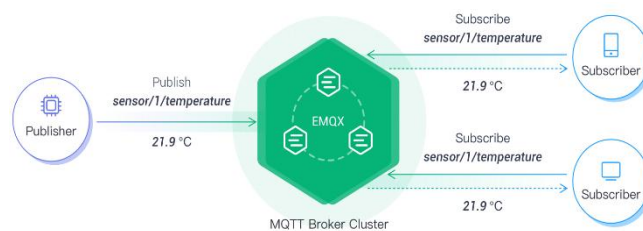


Figure 5 – Architecture MQTT : schéma Publisher/Broker/Subscriber.

5. Implémentation logicielle (Software)

5.1 Outils et technologies

- **Visual Studio Code** : IDE principal.
- **Flutter + Dart** : développement mobile.
- **Package Flutter `mqtt_client` (v10.0.0)** : client MQTT côté application.
- **Lib C++ `PubSubClient`** : client MQTT côté ESP32.
- **Broker public** : `broker.hivemq.com`.

5.2 Application mobile Flutter

5.2.1 Structure du projet

Les fichiers principaux de l'application se trouvent dans le dossier `lib/` :

- `main.dart` : point d'entrée, lance `MaterialApp`.
- `home_page.dart` : page d'accueil : connexion MQTT, affichage des statuts et liste des pièces.

- `room_page.dart` : page de commande d'une pièce (UP/DOWN + slider + message manuel).
- `Mqttpage.dart` : page de test

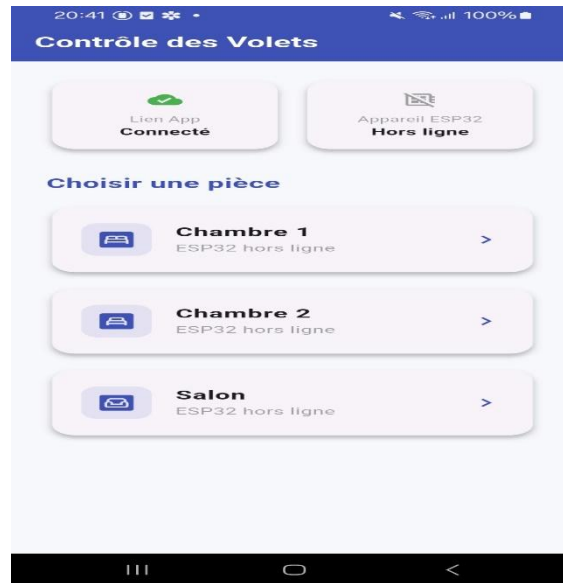


Figure 1 : Page d'accueil

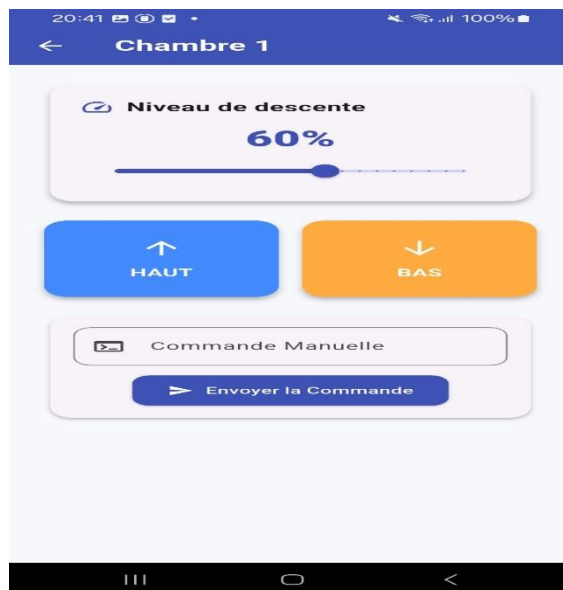


Figure 2 : Page d'une pièce UP/BAS + slider

5.2.2 Navigation et expérience utilisateur

L'application suit une navigation simple :

- l'utilisateur ouvre l'app → voit l'état de la liaison MQTT et l'état de l'ESP32,
- il sélectionne une pièce (Chambre 1, Chambre 2, Salon),
- il envoie des commandes via :
 - boutons directionnels (HAUT/BAS),
 - slider (0–100%),

- champ de commande manuelle

5.2.3 Connexion MQTT côté Flutter

La connexion est faite via `MqttServerClient` :

- broker : `broker.hivemq.com`
- port : 1883
- keep alive : 20 s
- session propre (*clean session*)

Une fois connecté, l'app :

- met à jour un état local `status = "Connecté"`,
- s'abonne au topic `esp32/status` (QoS *atLeastOnce*) pour afficher l'état « En ligne / Hors ligne ».

```
Future<void> connect() async {
  client = MqttServerClient(
    broker,
    'flutter_client_${DateTime.now().millisecondsSinceEpoch}',
  );
  client.port = 1883;
  client.keepAlivePeriod = 20;
  client.logging(on: false);

  client.connectionMessage = MqttConnectMessage()
    .withClientIdentifier(client.clientIdentifier)
    .startClean();

  try {
    await client.connect();
    setState(() => status = "Connecté");

    client.subscribe(statusTopic, MqttQos.atLeastOnce);
    client.updates!.listen((List<MqttReceivedMessage<MqttMessage>> c) {
      final MqttPublishMessage msg = c[0].payload as MqttPublishMessage;
      final String pt = MqttPublishPayload.bytesToStringAsString(
        msg.payload.message,
      ).trim();
    });
  } catch (e) {
    // Handle connection error
  }
}
```

Figure 3 – Connexion MQTT + abonnement statut

5.2.4 Affichage du statut ESP32

Le topic `esp32/status` reçoit par exemple :

- `online` → affiché « En ligne »
- `offline` → affiché « Hors ligne »

L'ESP32 publie `online` en message retenu (*retained*), ce qui permet à l'app de récupérer immédiatement le dernier état disponible.

5.2.5 Publication des commandes (UP/DOWN/%)

Sur `room_page.dart`, l'app publie une charge utile **texte** :

- bouton HAUT → publie "up"

- bouton BAS → publie "down"
- slider → publie la valeur 0 . . 100 sous forme de chaîne, envoyée lors de `onChangeEnd`.

Paramètres :

- QoS : *atLeastOnce*
- topic : celui de la pièce choisie (`esp32/roomX`).

```
void _sendMqttMessage(String message) {
    if (widget.client.connectionStatus?.state != MqttConnectionState.connected) {
        ScaffoldMessenger.of(context).showSnackBar(
            const SnackBar(content: Text("Non connecté au serveur MQTT")),
        );
        return;
    }

    final builder = MqttClientPayloadBuilder();
    builder.addString(message);
    widget.client.publishMessage(widget.topic, MqttQos.atLeastOnce, builder.payload!);
}

void sendMessage() {
    final text = controller.text.trim();
    if (text.isNotEmpty) {
        _sendMqttMessage(text);
        controller.clear();
    }
}
```

Figure 4 :Publication

5.2.6 Gestion des erreurs (application)

Avant chaque publication, l'app vérifie l'état de connexion MQTT. Si l'app n'est pas connectée, elle affiche un *SnackBar* :

- « Non connecté au serveur MQTT »

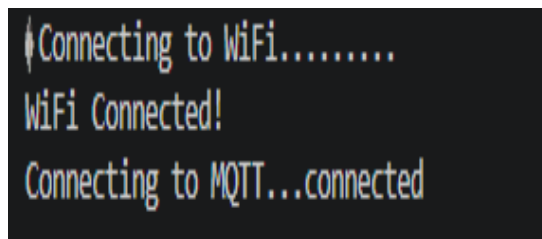
Cela améliore la robustesse et évite d'envoyer des commandes sans effet.

5.3 Firmware ESP32 (PlatformIO / C++)

Le firmware est implémenté dans `tryingmqtt/src/main.cpp`.

5.3.1 Connexion Wi-Fi

L'ESP32 se connecte au Wi-Fi via `WiFi.begin(ssid, password)` et boucle jusqu'à obtenir `WL_CONNECTED`.



```
Connecting to WiFi.....
WiFi Connected!
Connecting to MQTT...connected
```

Figure 8: Connecting to WiFi

5.3.2 Connexion MQTT + Last Will

La connexion MQTT (PubSubClient) est réalisée dans `reconnect()`. Le code configure un *Last Will* sur `esp32/status` :

- en cas de déconnexion inattendue : publication `offline`.

Quand la connexion réussit, l'ESP32 :

- s'abonne aux topics `esp32/room1`, `esp32/room2`, `esp32/room3`,
- publie `online` sur `esp32/status` avec le flag *retained*.

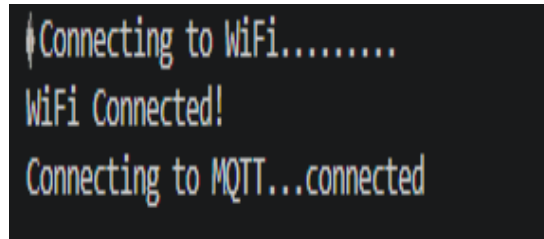


Figure 9: Connecting to MQTT

5.3.3 Contrôle bidirectionnel (Pins A/B)

Pour piloter une charge « montée/descente », on utilise deux sorties (A et B) par pièce :

- `room1PinA=26, room1PinB=27`
- `room2PinA=25, room2PinB=33`
- `room3PinA=32, room3PinB=14`

Logique de direction (exemple Chambre 1) :

- commande `up` → `A=HIGH, B=LOW`
- commande `down` → `A=LOW, B=HIGH`

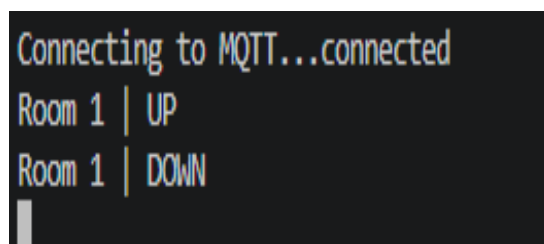


Figure 10 : Room 1 | UP et Room 1 | DOWN

5.3.4 Commande par pourcentage (0–100%) → durée 0–10 s

Quand l'ESP32 reçoit un message numérique `val` entre 0 et 100 sur `esp32/room1` :

- il calcule une durée : `moveTime = map(val, 0, 100, 0, 10000)`
- il active la direction d'ouverture
- il attend `moveTime` millisecondes

- il stoppe le mouvement

Cette commande permet une ouverture partielle simple (approche temporelle)

Cette stratégie correspond à notre hypothèse : ouverture complète = 10 s.

Pour visualiser l'exécution, une LED de mouvement est pilotée sur la pin 17 :

- LED17 ON pendant le mouvement
- LED17 OFF à l'arrêt

5.3.5 Boutons manuels (fallback)

Deux boutons locaux (contrôle manuel) sont câblés sur :

- pin 18 : commande montée (Room 1)
- pin 19 : commande descente (Room 1)

Ils sont configurés en `INPUT_PULLUP`. La détection se fait sur le front (transition HIGH→LOW). Une temporisation (`delay(3000)`) sert de filtrage simple et limite les répétitions.

6. Conception matérielle (Hardware)

6.1 Composants utilisés (prototype)

- **ESP32** : cœur du système embarqué.
- **Charges de test** : LEDs + résistances (ou relais / driver moteur selon matériel disponible).
- **Boutons poussoirs** : contrôle manuel local.
- **Breadboard + câbles** : prototypage.
- **Alimentation** : via USB (prototype).

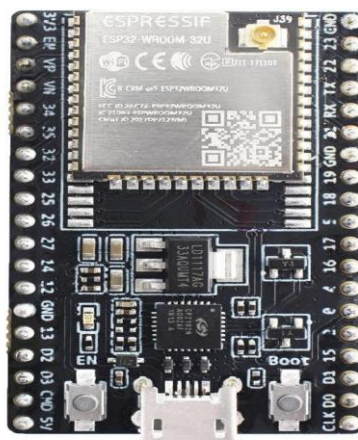
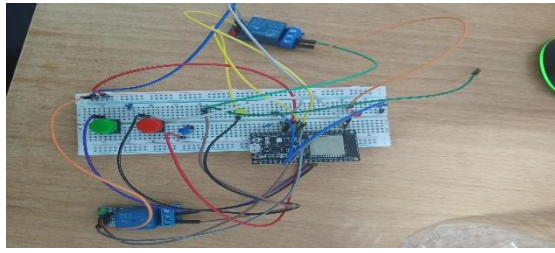


Figure 6 : CARTE ESP32.



(Figure 7) : photo du montage (LEDs, résistances, boutons, câblage).

6.2 Principe de commande d'un volet

Un volet roulant nécessite une commande bidirectionnelle :

- sens 1 : montée
- sens 2 : descente

Le prototype utilise deux sorties numériques (A/B) pour représenter ces deux sens. En conditions réelles, ces sorties peuvent commander :

- un **pont en H** (moteur DC), ou
- un **module relais double** (selon le moteur et le type de commande).

Point de sécurité : le choix exact des niveaux logiques (HIGH/LOW) et de l'état d'arrêt (coast/brake) dépend du matériel de puissance utilisé. Dans tous les cas, il faut éviter d'activer simultanément deux commandes opposées si l'interface de puissance ne le permet pas.

7. Expérimentation et tests

Test 1 – Compilation et upload ESP32 (PlatformIO)

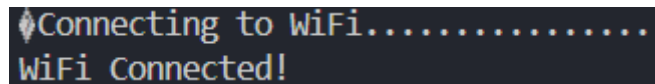
- **Objectif** : vérifier que le firmware compile et se téléverse correctement.
- **Procédure** :
 1. Ouvrir le projet PlatformIO.
 2. Lancer *Build* puis *Upload*.
- **Résultat attendu** : succès du build + upload.

```
Writing at 0x000b8a30... (93 %)
Writing at 0x000be428... (96 %)
Writing at 0x000c384b... (100 %)
Wrote 756400 bytes (489684 compressed) at 0x00010000 in 11.7 seconds (effective 516.4 kbit/s)...
Hash of data verified.

Leaving...
Hard resetting via RTS pin...
===== [SUCCESS] Took 17.77 seconds =====
* Terminal will be reused by tasks, press any key to close it.
```

Test 2 – Connexion Wi-Fi ESP32

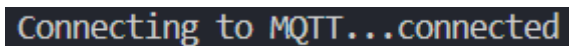
- **Objectif** : vérifier la connexion au réseau.
- **Procédure** : ouvrir le moniteur série après upload.
- **Résultat attendu** : affichage « WiFi Connected! ».



```
Connecting to WiFi.....  
WiFi Connected!
```

Test 3 – Connexion MQTT et abonnement

- **Objectif** : vérifier la connexion au broker et l'abonnement aux topics.
- **Procédure** :
 1. Démarrer l'ESP32.
 2. Observer le moniteur série.
- **Résultat attendu** : « Connecting to MQTT... connected ».



```
Connecting to MQTT...connected
```

Test 4 – Connexion de l'application et statut ESP32

- **Objectif** : vérifier la connexion MQTT côté Flutter et la réception de `esp32/status`.
- **Procédure** :
 1. Lancer l'app sur smartphone.
 2. Observer les cartes de statut.
- **Résultat attendu** :
 1. Lien App : « Connecté »
 2. Appareil ESP32 : « En ligne » (si ESP32 allumé)

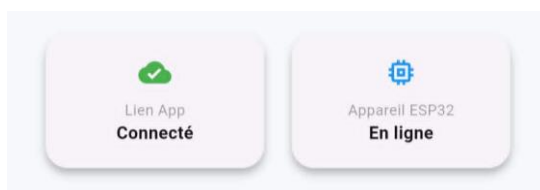
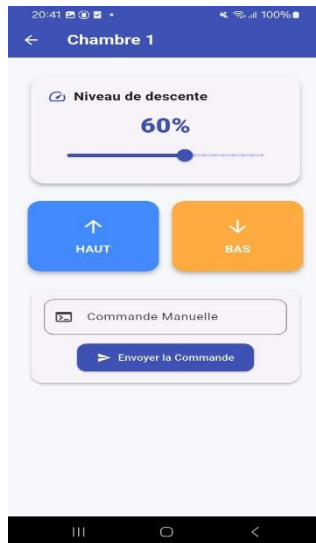


Figure 11 : client MQTT `esp32/status` = online.

Test 5 – Commande UP/DOWN (topic pièce)

- **Objectif** : vérifier la propagation App → MQTT → ESP32 → sorties GPIO.
- **Procédure** :
 1. Depuis l'app, entrer dans « Chambre 1 ».
 2. Appuyer sur **HAUT** puis **BAS**.
- **Résultat attendu** : logs :
 1. « Room 1 | UP »
 2. « Room 1 | DOWN »



```
Connecting to MQTT...connected
Room 1 | UP
Room 1 | DOWN
```

Test 6 – Commande par pourcentage

- **Objectif** : vérifier la conversion % → durée 0–10 s et l'arrêt automatique.
- **Procédure** :
 1. Mettre le slider à 65% et relâcher.
 2. Observer LED17 (mouvement) et le moniteur série.
- **Résultat attendu** :
 1. réception « Room 1 | Percent = 50 »
 2. LED17 ON pendant ~6.5 s puis OFF

```
Room 1 | Percent = 65
LED17 ON
LED17 OFF
```

Figure 13: slider à 50% + log LED17 ON/OFF.

Test 7 – Boutons manuels (fallback local)

- **Objectif** : vérifier le contrôle sans passer par l'application.
- **Procédure** : appuyer sur bouton pin18 puis pin19.

8. Conclusion et perspectives

Ce projet a permis de réaliser un prototype complet de commande de volets roulants basé sur l'IoT, en intégrant :

- une application Flutter moderne,
- une communication MQTT légère et efficace,
- un firmware ESP32 capable d'exécuter des commandes bidirectionnelles,
- un contrôle manuel local, utile en cas de problème réseau.

Perspectives d'amélioration

- Remplacer `delay()` par une gestion non bloquante (`millis()`), pour garder un MQTT fluide.
- Ajouter un vrai retour d'état (position estimée, fins de course, capteur de courant).
- Sécuriser la communication (TLS, authentification) et éviter les secrets en dur.
- Ajouter un broker local (Mosquitto sur Raspberry Pi) pour fonctionner hors Internet.

9. Références

- Documentation Flutter : <https://docs.flutter.dev/>
- Package `mqtt_client` : https://pub.dev/packages/mqtt_client
- MQTT (concepts) : <https://mqtt.org/>
- HiveMQ public broker : <https://www.hivemq.com/public-mqtt-broker/>

10. Annexes

Annexe A – Table topics / messages / pins

Élément	Topic	Messages	Pins ESP32	Action
Statut ESP32	<code>esp32/status</code>	<code>online / offline</code>	—	Indique l'état de la carte
Chambre 1	<code>esp32/room1</code>	<code>up, down, 0..100</code>	A=26, B=27, LED=17,	Direction + durée

Élément	Topic	Messages	Pins ESP32	Action
			BTN=18/19	
Chambre 2	esp32/room2	up, down, 0..100	A=25, B=33	Direction
Salon	esp32/room3	up, down, 0..100	A=32, B=14	Direction

Annexe B – Exemples de messages MQTT

- Topic: esp32/room1 / Payload: up
- Topic: esp32/room1 / Payload: down
- Topic: esp32/room1 / Payload: 50
- Topic: esp32/status / Payload: online